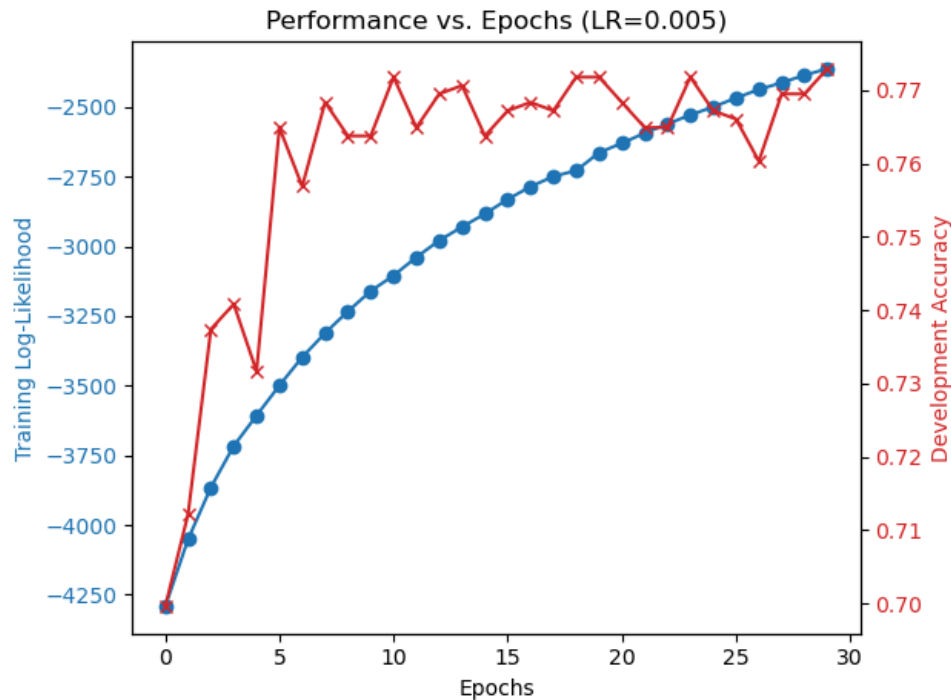


Part 1: Logistic regression

Hyperparameters: learning rate = 0.005, epochs = 30, SEED = 42

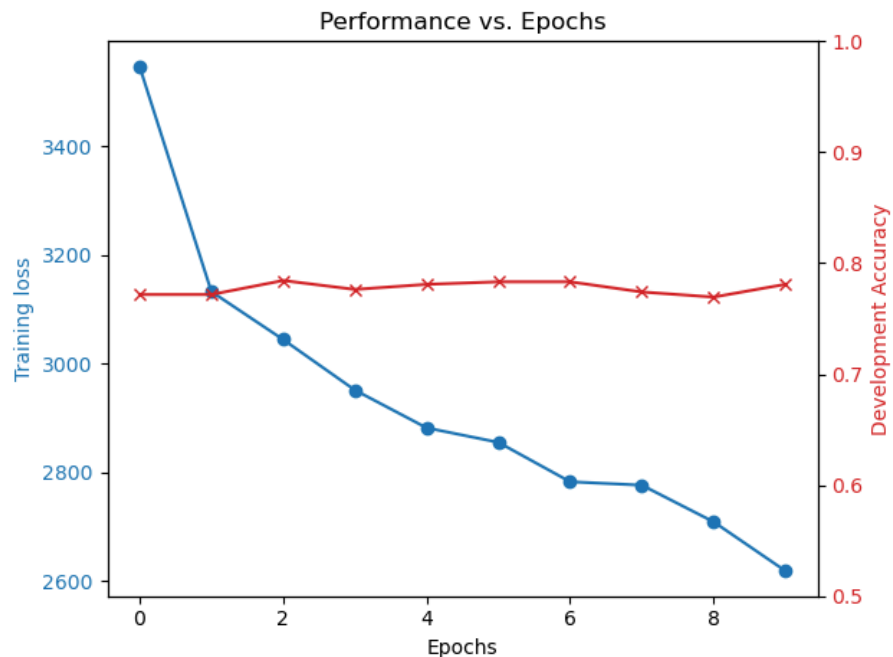


dev_acc: 0.7729, **dev_f1:** 0.7749, **execution_time:** 4.716,

Accuracy: 674 / 872 = 0.7729, **Precision:** 341 / 436 = 0.7821 **Recall:** 341 / 444 = 0.7680 **F1:** 0.7750

Part 2: Deep Averaging Network

Hyperparameters: learning rate = 0.001, epochs = 10, SEED = 42



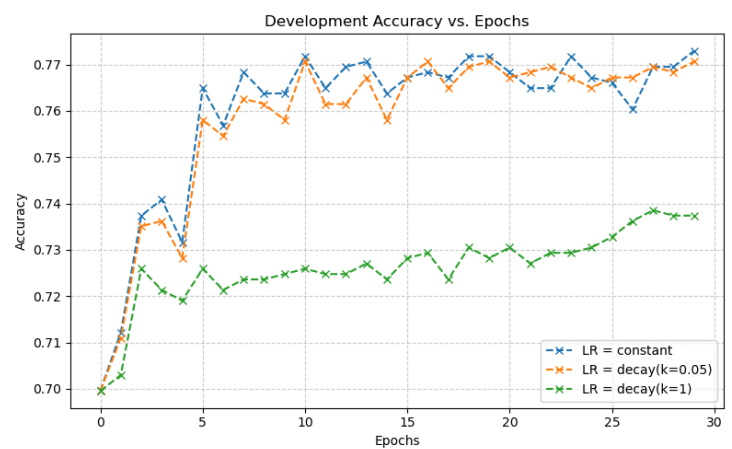
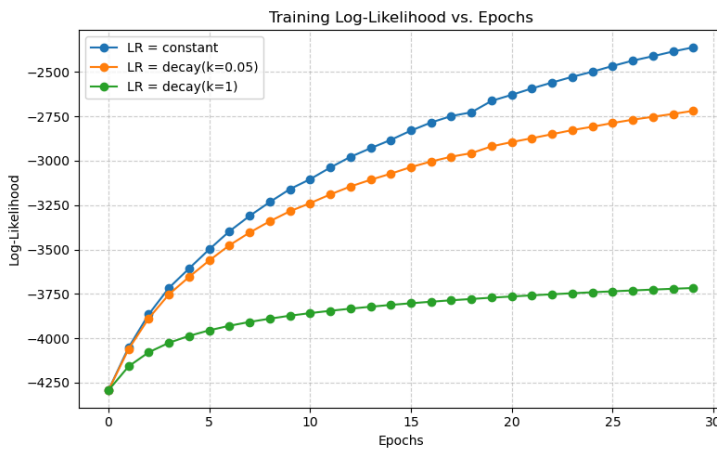
dev_acc: 0.7809, **dev_f1:** 0.7832, **execution_time:** 19.07,

Accuracy: 681 / 872 = 0.7809, **Precision:** 345 / 437 = 0.7894, **Recall:** 345 / 444 = 0.7770, **F1:** 0.7832

Part 1: Exploration (a) performance for different learning rates

Experimented with the following learning rates:

- Constant
- Decay with an inverse time decay rate. $lr_epoch = lr_start / (1 + K * epoch)$
 - $K = 1$: quick decay (at the end of 30 epochs the learning rate is reduced to only 3% of start)
 - $K = 0.05$: slow decay (at the end of 30 epochs the learning rate is reduced to 40% of start)



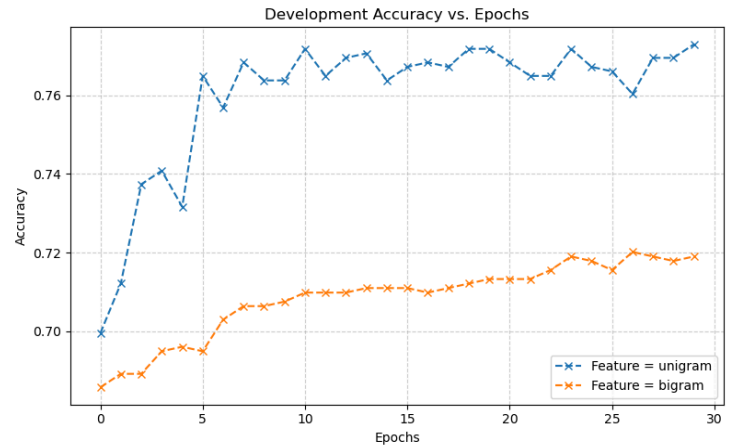
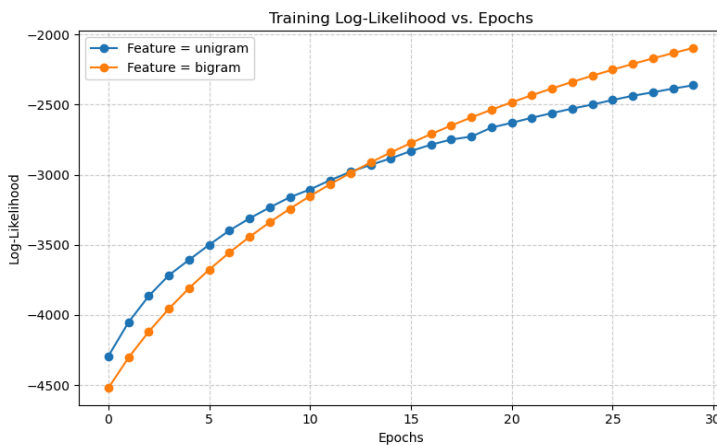
Observations:

1. Slow decay resulted in smoothing of the dev accuracy: Once the model is trained reducing learning rate could will result in slower updates to the parameters effectively smoothing the accuracy curve
2. Quick decay in learning rate significantly drops the accuracy: Model couldn't learn from the data and the learning rate dropped significantly over epochs reaching ~3% of the initial value.

Part 1: Exploration (b) BIGRAM and UNIGRAM comparison

Bigram-features:

Example: I like the movie: Features: {I|like: 1, like|the: 1, the|movie: 1}



Observations: Training log-likelihood is higher for BIGRAM compared to UNIGRAM however the dev_accuracy is significantly lower for BIGRAM which shows that the BIGRAM model has overfitted

Conclusion: Our corpus has far more unique BIGRAM than UNIGRAM resulting in overfitting. Hence for generalization the UNIGRAM features are better.